AutoDSC: Automated Algorithm Selection and Tuning for Non-Stationary Data Stream Clustering

Anonymous

***Abstract*—We introduce *AutoDSC* (Automated Data Stream Clustering), a novel AutoML framework that automatically selects and tunes clustering algorithms for evolving data streams. This matters because in IoT, network monitoring, and sensor applications, the data distribution changes continuously, so no fixed algorithm or configuration stays optimal, yet clustering lacks a universal objective to optimise, which is why AutoML has transformed supervised learning, yet its extension to unsupervised streams remains largely unexplored. *AutoDSC* achieves fully autonomous adaptivity by continuously searching the joint algorithm and hyperparameter space using population-guided stochastic search optimisation and selecting the best configuration at each window stage using a multi-objective quality measure, without ground-truth labels. We benchmark AutoDSC against state-of-the-art stream-clustering and AutoML methods on ten representative real-world and synthetic data streams. AutoDSC attains the best overall rank in clustering quality among all compared methods while remaining computationally efficient.**

***Index Terms*—data stream clustering, AutoML, algorithm selection, non-stationary, unsupervised learning, multi-objective optimisation**

I. Introduction

Data stream clustering is a fundamental operation across a wide range of real-world applications, from IoT sensor networks and network intrusion detection to social media analysis and financial monitoring [1], [2]. Unlike batch clustering, stream clustering must process data continuously, adapting to distributions that change over time. Numerous stream clustering algorithms have been proposed [3]–[6], each effective under specific conditions. However, the right algorithm and its hyperparameters must themselves change as the stream evolves, since no single configuration stays optimal throughout. Existing automated solutions either rely on a single evaluation metric [7], require computationally intensive offline tuning [8], or reconfigure structural components without jointly searching the algorithm and hyperparameter space [9], leaving a clear gap for a fully autonomous, label-free AutoML approach to stream clustering.

In this paper, we introduce *AutoDSC* (Automated Data Stream Clustering), a novel AutoML framework that automatically selects and tunes a clustering algorithm online as the data distribution evolves. We make the following key contributions.

- We formalise data stream clustering as a combined algorithm selection and hyperparameter optimisation (CASH) problem [10] that must be solved continuously as the data distribution evolves, and present *AutoDSC*, a novel AutoML framework that solves it autonomously without manual tuning or labels.
- *AutoDSC* introduces Population-Guided Stochastic Search Optimisation (PGSSO), which maintains a recency-weighted elite archive and an adaptively controlled exploration rate so that promising configurations are identified quickly and the population recovers implicitly when the distribution shifts.
- *AutoDSC* scores each configuration with a normalised multi-objective quality measure combining multiple internal cluster-validity indices, yielding robust selection.
- We provide a comprehensive empirical evaluation across ten real-world and synthetic streams, demonstrating that *AutoDSC* consistently ranks best in clustering quality and is the most computationally efficient among the data stream clustering AutoML frameworks[1].

The rest of the paper is organised as follows. Section II introduces the background of data stream clustering and AutoML. Section III formalises the problem. Section IV presents the *AutoDSC* framework. Section V describes the experimental setup. Section VI presents the empirical evaluation and analysis. Section II-C reviews prior work, and Section VIII concludes the paper.

II. Background

A. Data Stream Clustering

A data stream is an unbounded, high-velocity sequence of instances that a learner must process continuously. A stream learner must satisfy five requirements [11], namely to **RQ1** process one instance at a time and inspect it at most once; **RQ2** use a bounded amount of time per instance; **RQ3** use a bounded amount of memory; **RQ4** be ready to produce an answer at any time; and **RQ5** adapt to temporal changes in the data. Because the data-generating distribution evolves, streams exhibit *concept drift*, meaning the number, shape, and location of clusters changes over time, whether abruptly, gradually, or incrementally [11], [12], meaning a static configuration that performs well today may become suboptimal as conditions shift. Classical stream clustering algorithms address RQ1–RQ5 and fall into families distinguished by how they summarise the stream: CluStream [3] maintains pyramidal time-frame statistics; density-based methods DenStream [4] and DBSTREAM [5] discover arbitrarily shaped clusters via core, potential, and outlier micro-clusters; and k-means-based methods maintain centroids online, exemplified by mini-batch web-scale clustering [13] and StreamKMeans [6]. Stream clustering quality is evaluated with the prequential (interleaved

[1]Datasets and code: https://anonymous.4open.science/r/AutoDSC

test-then-train) protocol [11], in which each instance tests the current model before the model updates on it. Because ground-truth labels are unavailable during the stream, online quality assessment relies on *internal* cluster validity indices (CVIs), computed from the data and cluster assignments alone: the Silhouette Coefficient [14] measures cohesion relative to neighbouring clusters; the Davies–Bouldin Index [15] measures the ratio of intra-cluster scatter to inter-cluster separation; and the Calinski–Harabasz Index [16] measures the ratio of between-cluster to within-cluster dispersion. *External* indices require ground-truth labels for evaluation: the Adjusted Rand Index (ARI) [17], [18] is a chance-corrected measure of agreement between a predicted clustering and the true partition, yielding 1 for a perfect match and 0 for random assignment; and the Cluster Mapping Measure (CMM) [19] extends external evaluation to the streaming setting by explicitly accounting for emerging, splitting, and moving clusters that classical external indices treat as errors.

B. Automated Machine Learning

AutoML formalises algorithm selection and hyperparameter optimisation as the Combined Algorithm Selection and Hyperparameter Optimisation (CASH) problem [10], [20], [21], with search strategies ranging from Bayesian optimisation and evolutionary methods to population-based search and random search [22], [23]. Clustering-specific systems apply these strategies with meta-learning warm-starts [24]–[27], but all assume full dataset access and a fixed distribution. AutoML has been extended to supervised data streams [28]–[31], yet its extension to unsupervised streams, where no ground-truth labels are available to guide search, remains largely unexplored.

C. Prior Work

AutoML has been applied to batch clustering [24]–[27], [32], [33] and to supervised data streams [28]–[31], yet no existing system jointly addresses the single-pass, label-free, and continuously drifting conditions of stream clustering. The closest prior work spans three systems. *ConfStream* [7] maintains an ensemble of configurations and resamples near the better-performing ones, but selects by a single Silhouette-Width score that proves brittle across datasets. *OnlineAutoClust* (OACE) [8] searches with Tree-structured Parzen Estimator Bayesian optimisation over clustering ensembles, but depends on a computationally heavy offline tuning budget and is slow to recover under rapid drift. *MOStream* [9], the automated successor of the modular Benne algorithm from the SESAME study [34], reconfigures four structural design dimensions by rule rather than searching the joint algorithm-and-hyperparameter space. *AutoDSC* addresses all this limitations through a single-pass population-guided search over the joint algorithm-and-hyperparameter space, selection by a multi-objective label-free quality measure, and implicit drift adaptation through continual re-evaluation.

III. Problem Formulation

A data stream is an unbounded sequence $\mathcal{S} = \{x_1, x_2, \ldots, x_t, \ldots\}$ in which each instance $x_t \in \mathbb{R}^d$ arrives one at a time and is consumed in consecutive windows of fixed size w. Because ground-truth labels are unavailable online, clustering quality on window W is judged by an internal cluster-validity score $q(\cdot) \in [0, 1]$ computed from the data and assignments alone, while external indices are reserved for evaluation.

Let $\mathcal{A}$ be a portfolio of stream-clustering algorithms

$$\Theta = \bigcup_{a \in \mathcal{A}} (\{a\} \times \Phi_a) \tag{1}$$

the joint space of configurations $\theta = (a, \phi)$ pairing an algorithm a with hyperparameters $\phi \in \Phi_a$. The CASH problem for a stream is to track, at the end of each stage, the configuration

$$\theta^\star = \arg\max_{\theta \in \Theta} \; q(\theta; W), \tag{2}$$

using only the data seen so far and bounded memory. As the distribution drifts and $\theta^\star$ changes over time, so a solver must continually re-evaluate and re-select configurations.

IV. Automated Data Stream Clustering

AutoDSC addresses the online CASH problem of Section III as a novel AutoML framework tailored for non-stationary data stream clustering. *AutoDSC* processes data in fixed-size stages, evaluating clustering configurations at the end of each stage using multiple normalised validity indices. The framework maintains a recency-weighted elite archive of high-performing configurations to guide future exploration via population-guided stochastic search. Figure 1 illustrates the high-level workflow of *AutoDSC*.

The framework maintains a configuration population $\mathcal{A}_t \subseteq \Theta$ of clustering algorithms with different hyperparameter settings, fills the current window W_t as instances arrive, and serves real-time predictions from the best-performing (incumbent) configuration. As each new data point arrives, every configuration in the population updates incrementally on it with a single LearnOne step, retaining the per-point cluster assignment for scoring. At the end of each window, *AutoDSC* evaluates clustering quality using three normalised cluster validity indices, the Silhouette Coefficient (NSC), the Davies-Bouldin Index (NDBI), and the Calinski-Harabasz Index (NCHI). The expected clustering quality for configuration θ at time t is

$$\mu_t(\theta) = \frac{1}{3}[\text{NSC} + \text{NDBI} + \text{NCHI}], \tag{3}$$

where all indices are bounded to $[0, 1]$ with higher values indicating better clustering quality. Equal weights avoid dataset-specific tuning and are consistent with the finding that no single CVI dominates across cluster shapes and densities [35], [36].

AutoDSC adjusts exploration based on the change in the best score between consecutive stages, $\Delta\mu_t = \mu_t(\theta_t^*) - \mu_{t-1}(\theta_{t-1}^*)$. The exploration rate then adapts as:

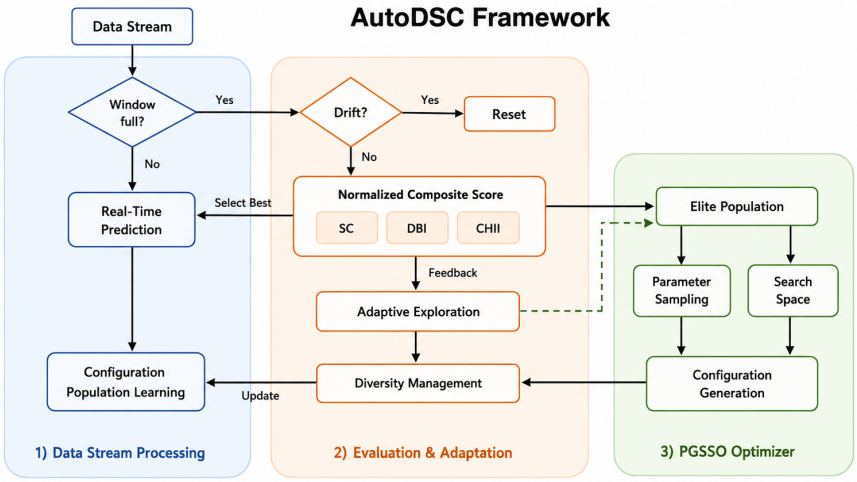


Fig. 1. Overview of AutoDSC. At the end of each stage the population of clustering configurations is evaluated by the multi-objective metrics, the incumbent is re-selected, and PGSSO proposes new configurations from the recency-weighted elite archive under an adaptive exploration rate.

$$\alpha_{t+1} = \begin{cases} \max(0.1,\ \alpha_t \cdot (1-\beta_t)) & \Delta\mu_t > 0 \\ \min(1.0,\ \alpha_t \cdot (1+\beta_t)) & \Delta\mu_t \le 0 \end{cases} \tag{4}$$

$$\beta_t = \min(\gamma,\ |r_t|) \tag{5}$$

where $r_t = \Delta\mu_t / |\mu_{t-1}(\theta^*_{t-1})|$ is the relative improvement and β_t is the resulting step, capped at γ for stability. This parameter-free mechanism decreases exploration when performance improves and increases it when performance stagnates or drops, as happens under concept drift, encouraging broader search to recover.

The framework employs **Population-Guided Stochastic Search Optimisation (PGSSO)** (Algorithm 2) to jointly optimise algorithm selection and hyperparameter tuning. PGSSO maintains an elite archive $\mathcal{E}_t = \{(\theta_i, s_i)\}_{i=1}^{e}$ where each configuration $\theta_i = (a_i, \phi_i)$ pairs an algorithm with its hyperparameters and $s_i = \mu_t(\theta_i)$ is its score. To remain responsive to drift, elites are ranked by a *recency-weighted* score $s_i/(1+\Delta_i)$, where Δ_i is the number of windows since θ_i was last scored, so stale configurations are down-weighted in favour of recently effective ones. PGSSO balances exploration and exploitation through dynamic rate α_t, computing statistical distributions over algorithm selections via frequencies and hyperparameters via means m_j and variances σ_j^2, which together guide sampling of new configurations.

PGSSO generates new configurations through probabilistic sampling:

$$\theta_{\text{new}} = \begin{cases} \text{Elite-Guided}(\mathcal{E}_t) & \text{with probability } (1-\alpha_t) \\ \text{Random}(\Theta) & \text{with probability } \alpha_t \end{cases} \tag{6}$$

Elite-guided sampling first draws an algorithm with probability equal to its frequency in the elite archive, then samples that algorithm's numerical hyperparameters from a Gaussian fitted to the elites using it:

$$\phi_j \sim \mathcal{N}(m_j, \sigma_j^2) \tag{7}$$

where the mean m_j and variance σ_j^2 are computed over the elites that use the selected algorithm. Samples outside a hyperparameter's defined bounds are clipped to the boundary.

When the population exceeds size K, it is pruned by a farthest-first rule; the incumbent and top-scoring configurations are retained, then the most distant remaining configurations are added greedily to preserve diversity. The distance between two configurations is 1 if their algorithms differ and the mean normalised per-hyperparameter distance otherwise. *AutoDSC* emits a clustering for each instance using the incumbent θ^*_t. Algorithm 1 details the complete operational steps.

A. Computational Complexity

Table I summarises the per-instance and per-stage time and space complexity of AutoDSC, where K is the population size, C the base-clusterer cost per instance, w the stage size, d the feature dimensionality, p the number of parameters per configuration, and $e_{\max}{=}\lfloor K/2 \rfloor$ the elite archive capacity. Per-instance cost is $O(K \cdot C)$ regardless of stream length since K is fixed. The K^2 term in the per-stage cost accounts for diversity-based population pruning.

V. Experimental Setup

A. Datasets

We evaluate on ten datasets comprising five real-world and five synthetic streams with varying sizes, dimensionalities, and cluster counts, as summarised in Table II. Real-world streams cover those most frequently used in prior stream clustering work, while synthetic streams probe specific drift mechanisms in isolation through controlled generators.

Algorithm 1: AutoDSC

Input: Stream $\mathcal{S} = \{x_1, x_2, \ldots\}$, $x_t \in \mathbb{R}^d$; window size w; population size K
Output: Cluster predictions $\{y_t\}_{t=1}^{\infty}$ using θ_t^*

$e_{\max} \leftarrow \lfloor K/2 \rfloor$;
$\mathcal{E}_0 \leftarrow \emptyset$, $\alpha_0 \leftarrow 0.5$;
$\mathcal{A}_0 \leftarrow \{\theta_i\}_{i=1}^{K}$ where $\theta_i \sim \text{Uniform}(\Theta)$;
$W_0 \leftarrow \emptyset$, $\theta_0^* \leftarrow \emptyset$;
$\mu_{-1}(\theta_{-1}^*) \leftarrow -\infty$;
foreach $x_t \in \mathcal{S}$ **do**
  $W_t \leftarrow W_{t-1} \cup \{x_t\}$;
  foreach $\theta_i \in \mathcal{A}_t$ **do**
    $\ell_{i,t} \leftarrow \textsc{PredictOne}(M_i, x_t)$;
    `// store assignment`
    $M_i \leftarrow \textsc{LearnOne}(M_i, x_t)$;
  if $|W_t| = w$ **then**
    foreach $\theta_i \in \mathcal{A}_t$ **do**
      $L_i \leftarrow \{\ell_{i,\tau} : x_\tau \in W_t\}$;
      $\mu_t(\theta_i) \leftarrow \textsc{NormCVI}(W_t, L_i)$;
    $\mu_{\text{best}}^{(t)} \leftarrow \max_i \mu_t(\theta_i)$;
    $\alpha_{t+1} \leftarrow \textsc{Adapt}(\alpha_t, \mu_{t-1}(\theta_{t-1}^*), \mu_{\text{best}}^{(t)})$;
    $\mathcal{C}_{\text{new}} \leftarrow \text{PGSSO}(\{(\theta_i, \mu_t(\theta_i))\}, \alpha_{t+1}, \Theta)$;
    $\mathcal{A}_{t+1} \leftarrow \textsc{SelectDiverse}(\mathcal{A}_t \cup \mathcal{C}_{\text{new}}, K)$;
    $\theta_t^* \leftarrow \arg\max_{\theta_i \in \mathcal{A}_{t+1}} \mu_t(\theta_i)$;
    $W_t \leftarrow \emptyset$;

Algorithm 2: PGSSO

Input: Current configurations with scores $\{(\theta_i, s_i)\}$; exploration rate $\alpha \in [0, 1]$; parameter space Θ
Output: New configuration set $\mathcal{C}_{\text{new}}$

$\mathcal{E} \leftarrow \textsc{UpdateElitePopulation}(\{(\theta_i, s_i)\}, e_{\max})$;
`// recency-weighted ranking`
$\mathcal{E} \leftarrow \textsc{UpdateParameterDistributions}(\mathcal{E})$;
`// fit` $\mathcal{N}(m_j, \sigma_j^2)$ `per algorithm`
$\mathcal{C}_{\text{new}} \leftarrow \emptyset$;
for $j \leftarrow 1$ **to** n_{gen} **do**
  $u \leftarrow \text{Uniform}(0, 1)$;
  if $u > \alpha$ **then**
    $\theta_{\text{new}} \leftarrow \textsc{SampleFromElite}(\mathcal{E}, \Theta)$;
  else
    $\theta_{\text{new}} \leftarrow \textsc{RandomSample}(\Theta)$;
  $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C}_{\text{new}} \cup \{\theta_{\text{new}}\}$;
return $\mathcal{C}_{\text{new}}$;

The five real-world streams are public benchmarks with natural class structure and distribution change.

- **Insects** [37]: optical-sensor wing-beat features of flying insects, where the species labels and feature distribution shift with ambient temperature over the recording.
- **KDD99** [38]: network-connection records from the KDD Cup 1999 intrusion-detection task, labelled as normal traffic or one of several attack types.
- **PenDigits** [38]: pen-trajectory features of handwritten digits, with one cluster per digit 0–9.
- **Shuttle** [38]: NASA Space Shuttle radiator sensor readings (Statlog), with highly imbalanced operational-state classes.
- **Vehicle** [39]: high-dimensional acoustic and seismic features from a distributed wireless sensor network, labelled by vehicle type.

TABLE I
PER-INSTANCE AND PER-STAGE COMPLEXITY OF AUTODSC.

Complexity	Per-Instance	Per-Stage
Time	$O(K \cdot C)$	$O(K \cdot w \cdot d + K^2)$
Space	$O((K + e_{\max}) \cdot p \ + \ w \cdot d)$	

TABLE II
SUMMARY OF DATASETS.

Dataset Name	Type	Instances	Attributes	Clusters
Insects	Real	52,848	33	6
KDD99	Real	100,655	38	12
PenDigits	Real	10,992	16	10
Shuttle	Real	58,000	9	7
Vehicle	Real	98,528	100	3
Expanding	Synth	100,000	5	5
Gradual	Synth	100,000	5	5
Merge-Split	Synth	100,000	5	5
Sudden	Synth	100,000	5	5
Fast-RBF	Synth	100,000	5	5

The five synthetic streams isolate specific drift mechanisms under controlled conditions, holding dimensionality (d=5) and cluster count (k=5) fixed. Four are Gaussian-blob streams where each cluster c_i at time t generates points $x \sim \mathcal{N}(\mu_i(t), \sigma_i(t)^2 I)$, differing only in how $\mu_i(t)$ and $\sigma_i(t)$ evolve.

- **Expanding**: centroids are fixed ($\mu_i(t)$=μ_i^0); within-cluster standard deviation grows linearly, $\sigma_i(t) = \sigma_0 + \lambda t$, simulating continuous cluster expansion.
- **Gradual**: each centroid translates at constant velocity along a fixed random direction, $\mu_i(t) = \mu_i^0 + v_i t$, producing smooth continuous drift throughout the stream.
- **Merge-Split**: centroids move toward the global mean $\bar{\mu}$ during the first half of the stream and symmetrically re-separate in the second half, producing a single merge–split event.
- **Sudden**: centroid positions jump instantaneously at three evenly spaced change points and remain stationary between them, producing four stationary segments separated by abrupt concept drift.
- **Fast-RBF**: a MOA RandomRBF stream [40] with a high centroid speed parameter, generating continuously drifting clusters with complex shapes and overlapping boundaries.

TABLE III
SPECIFICATION OF OUR EVALUATION PLATFORM.

Component	Description
Processor	2× AMD EPYC 7702 64-Core @ 2.0 GHz (128 cores)
L3 Cache	512 MiB
Memory	1 TiB DDR4 RAM
OS	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-101-generic
Runtime	Python 3.12.3, River [6]
Compiler	GCC 13.3.0

B. Evaluation Protocol

All experiments use the prequential protocol with multiple independent runs using random seeds ranges 1-100 to account for stochastic variability. Clustering quality is measured using ARI [17] and CMM [19], where ground truth labels are used solely for evaluation, not training or model selection. We also record throughput (instances/s) and memory usage (MB) to assess computational efficiency. Statistical significance is assessed using the Friedman test followed by the Nemenyi post-hoc test [41] with $\alpha = 0.05$.

C. Baseline Methods Selection

We compare *AutoDSC* against five baselines spanning three categories.

- **AutoML:** *ConfStream* [7] and *OACE* [8], established frameworks that optimise algorithm selection and hyperparameters online.
- **Rule-based:** *MOStream* [9], which reconfigures structural components by rule without searching the joint algorithm-and-hyperparameter space.
- **Baselines:** *RandomSearch* [22], which shares *AutoDSC*'s framework structure but generates configurations by purely random sampling, and *Default(Best)* [8], where one of the most commonly-used approaches by non-expert users is to try all possible clustering algorithms with their default hyperparameters and then select the model with the best performance.

D. Implementation and Configuration

Following standard AutoML benchmarking practice [42], *AutoDSC* uses a population size $K = 10$ and window size $w = 1000$, with the internal parameter exploration rate initialised $\gamma_0 = 0.5$ to balance exploration and exploitation equally at the start, adapting automatically as performance feedback arrives. All AutoML baselines search over the same algorithm portfolio and hyperparameter ranges specified in Table IV, with window-dependent ranges listed for $w = 1000$. To ensure a fair comparison, so differences in results reflect search and selection strategy rather than compute allotment; other baseline-specific parameters remain as reported in their original papers. All experiments were conducted on the platform detailed in Table III, with datasets and code publicly available[2].

[2]https://anonymous.4open.science/r/AutoDSC

TABLE IV
SEARCH SPACE CONFIGURATION.

Algorithm	Hyperparameter	Type	Range/Options
CluStream	n_macro_clusters	integer	[2, 15]
	max_micro_clusters	integer	[20, 100]
	time_window	integer	[250, 1500]
	micro_cluster_r_factor	numeric	[1.0, 3.0]
DenStream	decaying_factor	numeric	[0.01, 0.2]
	beta	numeric	[0.2, 0.8]
	mu	numeric	[2.0, 8.0]
	epsilon	numeric	[0.01, 1.0]
DBSTREAM	clustering_threshold	numeric	[0.1, 2.0]
	fading_factor	numeric	[0.001, 0.05]
	intersection_factor	numeric	[0.1, 0.8]
	minimum_weight	numeric	[0.5, 5.0]
StreamKMeans	chunk_size	integer	[100, 500]
	n_clusters	integer	[2, 20]
	halflife	numeric	[0.05, 0.8]

VI. RESULTS

A. Clustering Quality

Table V compares *AutoDSC* against all baselines on clustering quality across both ARI and CMM. *AutoDSC* attains the best mean rank on both metrics and the highest mean scores across all methods, with the largest advantages on streams requiring continuous adaptation. On drifting synthetic streams such as Expanding, Gradual, and Fast-RBF, *AutoDSC* sustains near-perfect ARI and CMM while fixed-configuration baselines degrade substantially, demonstrating that per-stage re-evaluation and adaptive exploration are effective under persistent drift. A similar advantage appears on the harder real-world stream PenDigits, where *AutoDSC* leads on both ARI and CMM and no single algorithm stays competitive throughout. On streams where a single well-matched default algorithm is already near-optimal (Shuttle, Vehicle), the CMM advantage narrows since cluster structure is stable, and on Merge-Split, OACE leads narrowly on both metrics owing to its Bayesian optimisation history accumulated before the merge event. The prequential trajectories in Figure 2 confirm that *AutoDSC* stays at or above all baselines on both ARI and CMM consistently throughout both streams, reflecting stable adaptation rather than isolated performance spikes.

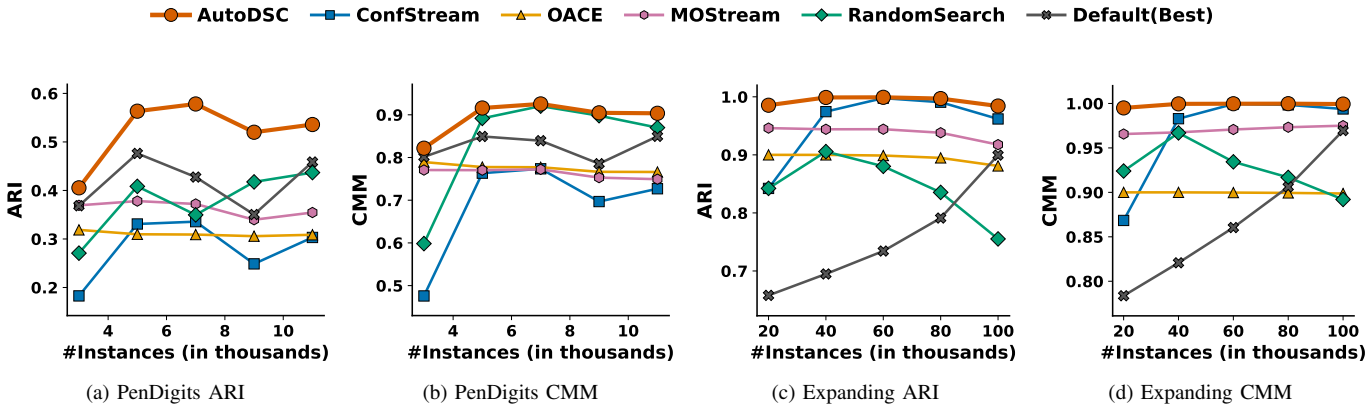
B. Drift Behaviour

Figure 3 isolates *AutoDSC*'s behaviour under two distinct drift types on synthetic streams. On Merge-Split, which contains a discrete merge-then-split event mid-stream, every adaptive method incurs a temporary quality drop and recovers over subsequent windows; *AutoDSC* sustains the highest stable ARI and CMM on either side of the drift event and recovers as quickly as the strongest baselines. On expanding, where within-cluster variance grows continuously, *AutoDSC* maintains near-perfect ARI and CMM for almost the entire stream and degrades the least as the drift intensifies, while ConfStream destabilises sharply in the later windows and OACE and MOStream plateau at lower quality throughout. *AutoDSC* therefore both recovers from abrupt drift and sustains quality under continuous drift.

TABLE V

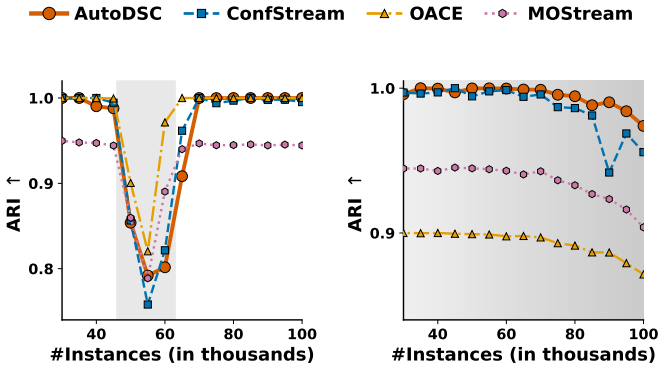
CLUSTERING QUALITY ACROSS DATASETS. ARI AND CMM (↑ HIGHER IS BETTER); BEST PER METRIC PER ROW IN **BOLD**, SECOND BEST UNDERLINED.

Dataset	AutoML						Rule-based		Baselines			
	AutoDSC		ConfStream		OACE		MOStream		RandomSearch		Default(Best)	
	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM
Insects	**.117±.003**	.905±.024	.016±.004	.827±.019	.048±.035	**.969±.028**	.090±.045	.964±.006	.054±.007	.880±.019	.100±.002	.947±.001
KDD99	**.037±.000**	.942±.015	.037±.001	.910±.013	.036±.003	.626±.133	.018±.014	.780±.174	.033±.001	.935±.017	.037±.000	**.990±.000**
PenDigits	**.510±.012**	**.888±.014**	.271±.029	.668±.025	.311±.120	.777±.107	.363±.010	.764±.016	.367±.035	.814±.016	.412±.000	.823±.000
Shuttle	.363±.011	.900±.006	.341±.002	.887±.000	.146±.083	**.974±.034**	.002±.001	.380±.054	.270±.014	.903±.018	**.409±.000**	.886±.000
Vehicle	.320±.002	.975±.005	.303±.001	.967±.000	.312±.003	.976±.000	.335±.017	**.990±.001**	.172±.017	.977±.001	**.335±.003**	.986±.000
Expanding	**.993±.001**	**.999±.001**	.953±.028	.969±.020	.895±.298	.900±.300	.938±.111	.970±.058	.844±.017	.927±.011	.756±.103	.868±.055
Gradual	**.998±.001**	1.000±.000	.956±.042	.964±.032	.995±.014	**1.000±.000**	.978±.066	.984±.047	.921±.014	.978±.004	.759±.130	.838±.094
Merge-Split	.963±.004	.988±.001	.952±.009	.979±.007	**.985±.001**	**.999±.000**	.932±.102	.968±.062	.877±.012	.947±.009	.895±.054	.945±.034
Sudden	**.921±.001**	**.956±.001**	.904±.013	.939±.010	.855±.003	.884±.000	.774±.090	.841±.067	.878±.006	.947±.002	.646±.160	.757±.116
Fast-RBF	**.999±.001**	**1.000±.000**	.988±.001	.989±.000	.995±.008	.999±.002	.911±.109	.938±.076	.924±.009	.963±.005	.818±.096	.896±.056
Mean Score	**.622**	**.955**	.572	.910	.558	.910	.534	.858	.534	.927	.517	.894
Mean Rank	**1.40**	**2.20**	3.60	4.30	3.60	2.90	4.10	3.80	4.50	3.50	3.80	4.30



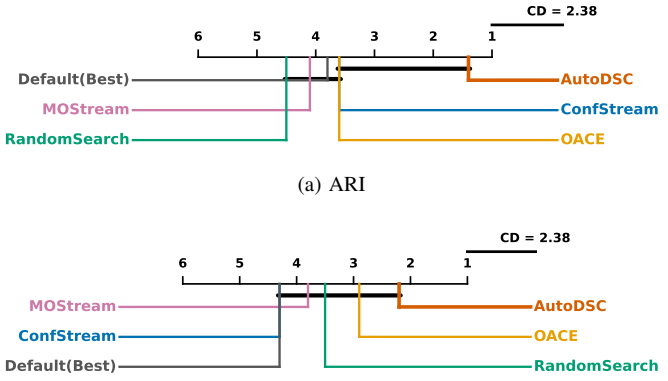
(a) PenDigits ARI (b) PenDigits CMM (c) Expanding ARI (d) Expanding CMM

Fig. 2. Prequential Clustering Quality Across Stream Stages. Results for all methods on a real (PenDigits) and a synthetic drifting (Expanding) stream.



(a) *Merge-Split* (abrupt) (b) *Expanding* (continuous)

Fig. 3. Drift behaviour on synthetic streams. The shaded region marks the drift interval.



(a) ARI

(b) CMM

Fig. 4. Critical Difference Diagrams for ARI and CMM. Ranks from the Friedman test with the Nemenyi post-hoc procedure ($\alpha = 0.05$).

C. Statistical Significance

Figure 4 shows the Critical Difference (CD) diagrams from the Friedman test with Nemenyi post-hoc procedure ($\alpha = 0.05$) applied to both ARI and CMM. *AutoDSC* achieves the best mean rank on both metrics. On ARI, *AutoDSC* is significantly better than RandomSearch, MOStream, and Default(Best). On CMM, no pair of methods reaches statistical significance, though *AutoDSC* nonetheless ranks first consistently across the evaluation.

D. Computational Efficiency

Figure 5 shows that *AutoDSC* occupies the favourable high-throughput, low-memory region of the efficiency plane, pairing the best clustering quality with a compact memory footprint. Among the AutoML methods, it is the fastest and the most memory-efficient, processing substantially more instances per second than ConfStream and OACE while consuming less memory than both. The Default(Best) baseline is naturally faster since it performs no search, and RandomSearch achieves

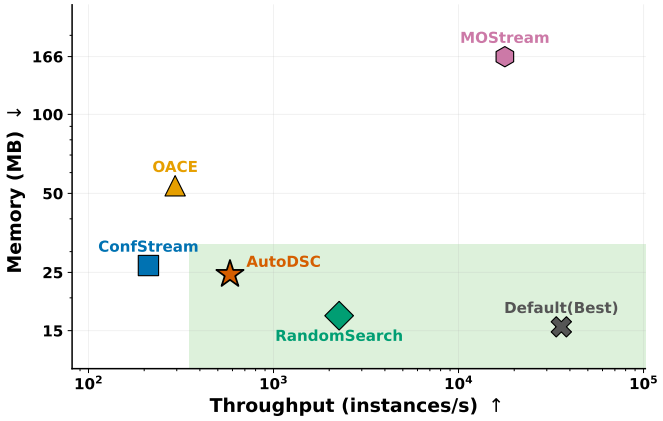


Fig. 5. Throughput-memory trade-off averaged across all datasets. The shaded green region marks the favourable high-throughput, low-memory area.

moderate speed at the cost of quality; *AutoDSC* achieves a better quality-efficiency balance than either. OACE incurs the heaviest AutoML memory footprint, driven by its offline Bayesian optimisation budget, while MOStream carries the largest footprint overall owing to its structural maintenance overhead.

E. Ablation Study

Figure 6 reports a leave-one-out ablation across all datasets in which one mechanism is disabled at a time, covering recency-weighted elite ranking, the adaptive exploration rate, the multi-objective CVI score, and diversity-preserving pruning, where replacing the guided PGSSO search with purely random generation is already captured by the RandomSearch baseline. The effects are small and complementary across the two metrics, where removing recency weighting produces the largest drop in ARI, while removing the multi-objective CVI score reduces CMM more than ARI, indicating that these two components each protect a different aspect of quality. Diversity pruning provides a consistent gain on both metrics, and the adaptive exploration rate shows a modest standalone contribution on stable streams, consistent with its primary role being drift recovery on the synthetic streams in Figure 3. The full configuration achieves the best balance across both ARI and CMM, supporting the value of the integrated design.

F. Population Evolution

Figure 7 shows how *AutoDSC* reshapes its population composition across stream windows on the Insects dataset, a real-world stream with pronounced concept drift. Rather than committing to a single algorithm, *AutoDSC* continuously shifts the composition between CluStream, DenStream, DBSTREAM, and StreamKMeans as the active cluster structure evolves. This emergent algorithm-switching behaviour arises purely from the recency-weighted population update, without any explicit drift detector.

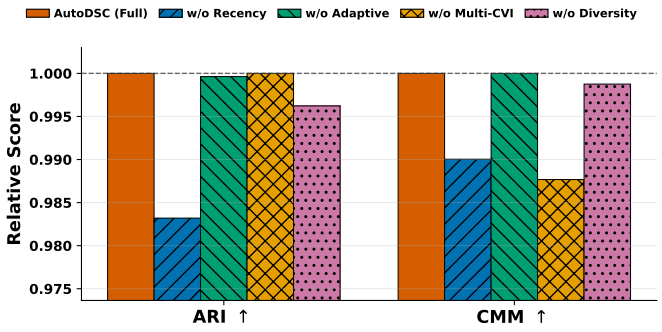


Fig. 6. Leave-one-out ablation across all datasets. ARI and CMM of each variant relative to full *AutoDSC*.

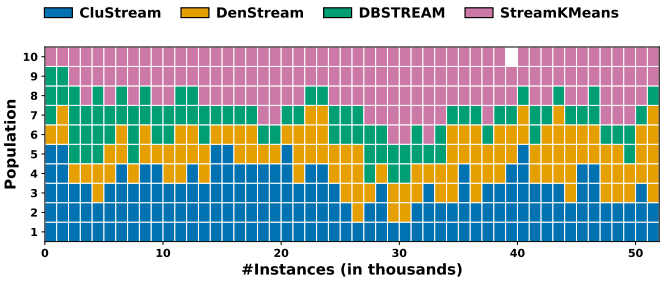


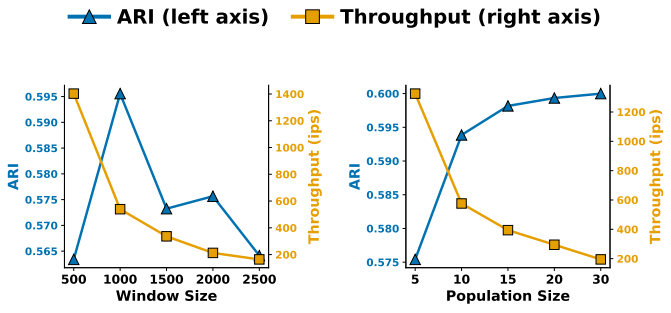
Fig. 7. Population Composition over the Insects Stream. Colors denote the selected base algorithm per population slot (rows) and stream stage (columns).

G. Sensitivity Analysis

Figure 8 reports *AutoDSC*'s sensitivity to its two free hyperparameters, window size w and population size K, averaged over all datasets. For window size, quality peaks at $w = 1000$, where smaller stages give insufficient data for reliable per-stage CVI evaluation and larger stages delay adaptation to drift and reduce throughput; the quality curve is thus unimodal with a clear optimum. For population size, both ARI and CMM rise steeply from small K and saturate around $K \approx 15$, while throughput falls monotonically, so the default $K = 10$ sits at the knee of the quality-efficiency trade-off. Overall, *AutoDSC* is robust within a broad range of both parameters and degrades gracefully outside it, with the default configuration ($w = 1000$, $K = 10$) providing the best balance between clustering quality and computational efficiency.

VII. Discussion and Limitations

AutoDSC advances AutoML for streaming environments through several design choices. Its multi-objective scoring function, combining three normalised cluster-validity indices, prevents convergence to configurations that excel on one aspect while failing on others [35], [36], and PGSSO's adaptive exploration rate enables rapid recovery after concept drift while exploiting good configurations during stable periods. *AutoDSC*'s clearest advantages are full autonomy, requiring no manual tuning or labels, and computational efficiency, together with statistically significant gains over untuned single algorithms and random search; its value is therefore best



(a) Window Size (b) Population Size

Fig. 8. Sensitivity to Window Size and Population Size. Averaged over all datasets.

framed as delivering competitive quality fully automatically and at lower cost rather than as a uniform quality win. These contributions bridge traditional AutoML and the dynamic requirements of real-time streaming applications in IoT, sensor networks, and analytics platforms, where manual configuration is impractical [2].

Several limitations bound the scope of our claims. First, with only ten datasets, the quality advantage of *AutoDSC* over the strongest AutoML competitors, ConfStream and OACE, is not statistically significant, although *AutoDSC* attains the best mean rank and significantly outperforms untuned single algorithms and random search. Second, our evaluation targets tabular streams with numerical features, so extension to sparse text, multimodal, or image streams would require domain-specific base clusterers. Third, *AutoDSC* still exposes two free hyperparameters, the window size and the population size, which require sensible defaults, although it remains stable across a broad range of both. Fourth, while PGSSO supports categorical hyperparameter sampling, the current portfolio uses only continuous and integer parameters, so algorithms with categorical choices would exercise this capability directly. Finally, configuration selection relies on internal validity indices computed without labels, so clustering quality is estimated rather than observed online, and a misleading internal score can temporarily affect selection between ground-truth evaluations.

VIII. Conclusion and Future Work

We presented *AutoDSC*, a single-pass AutoML framework for data stream clustering that proposes configurations with PGSSO and selects among them with a label-free multi-objective measure, adapting implicitly as the stream drifts. Across ten streams, *AutoDSC* attains the best mean rank on both ARI and CMM, with statistically significant gains over untuned single algorithms and random search, while remaining the fastest and most memory-efficient AutoML stream clusterer. These results demonstrate that fully autonomous, label-free algorithm selection and hyperparameter optimisation are both feasible and effective for evolving data streams, closing a gap that neither batch AutoML nor supervised stream AutoML addressed. Future work includes semi-supervised extensions and richer multi-objective optimisation objectives.

References

[1] L. Yang and A. Shami, "Iot data analytics in dynamic environments: From an automated machine learning perspective," *Engineering Applications of Artificial Intelligence*, vol. 116, p. 105366, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197622003803

[2] M. Bahri, A. Bifet, J. Gama, H. M. Gomes, and S. Maniu, "Data stream analysis: Foundations, major tasks and tools," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 11, no. 3, p. e1405, 2021.

[3] C. C. Aggarwal, S. Y. Philip, J. Han, and J. Wang, "A framework for clustering evolving data streams," in *Proceedings 2003 VLDB conference*. Elsevier, 2003, pp. 81–92.

[4] F. Cao, M. Ester, W. Qian, and A. Zhou, "Density-based clustering over an evolving data stream with noise," in *SIAM International Conference on Data Mining (SDM)*, 2006. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611972740.9

[5] M. Hahsler and M. Bolaños, "Clustering data streams based on shared density between micro-clusters," *IEEE Transactions on Knowledge and Data Engineering*, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7425260

[6] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem *et al.*, "River: machine learning for streaming data in python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

[7] M. Carnein, H. Trautmann, A. Bifet, and B. Pfahringer, "confstream: Automated algorithm selection and configuration of stream clustering algorithms," in *International conference on learning and intelligent optimization*. Springer, 2020, pp. 80–95.

[8] R. El Shawi and D. Rozgonjuk, "Onlineautoclust: A framework for online automated clustering," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 3870–3874.

[9] Z. Wang, X. Wang, and S. Zhang, "MOStream: A modular and self-optimizing data stream clustering algorithm," in *2024 IEEE International Conference on Data Mining (ICDM)*, 2024, pp. 500–509.

[10] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[11] A. Bifet, R. Gavaldà, G. Holmes, and B. Pfahringer, *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, 2018. [Online]. Available: https://moa.cms.waikato.ac.nz/book/

[12] C. C. Aggarwal, "Data streams: models and algorithms," *Springer Science & Business Media*, 2013.

[13] D. Sculley, "Web-scale k-means clustering," in *WWW 2010, Raleigh, North Carolina, USA*, 2010. [Online]. Available: http://code.google.com/p/sofia-ml

[14] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[15] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 1, no. 2, pp. 224–227, 1979.

[16] T. Caliński and J. Harabasz, "A dendrite method for cluster analysis," *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974.

[17] L. Hubert and P. Arabie, "Comparing partitions," *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[18] W. M. Rand, "Objective criteria for the evaluation of clustering methods," *Journal of the American Statistical association*, vol. 66, no. 336, pp. 846–850, 1971.

[19] H. Kremer, P. Kranen, T. Jansen, T. Seidl, A. Bifet, G. Holmes, and B. Pfahringer, "An effective evaluation measure for clustering on evolving data streams," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '11. New York, NY, USA: Association for Computing Machinery, 2011, pp. 868–876. [Online]. Available: https://doi.org/10.1145/2020408.2020555

[20] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013, pp. 847–855.
[21] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2962–2970.
[22] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *The journal of machine learning research*, vol. 13, no. 1, pp. 281–305, 2012.
[23] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
[24] D. Tschechlov, M. Fritz, and H. Schwarz, "Automl4clust: Efficient automl for clustering analyses," in *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2021, pp. 343–348.
[25] D. Treder-Tschechlov, M. Fritz, H. Schwarz, and B. Mitschang, "Ml2dac: Meta-learning to democratize automl for clustering analysis," *Proc. ACM Manag. Data*, vol. 1, no. 2, Jun. 2023. [Online]. Available: https://doi.org/10.1145/3589289
[26] R. ElShawi, H. Lekunze, and S. Sakr, "csmartml: A meta learning-based framework for automated selection and hyperparameter tuning for clustering," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 1119–1126.
[27] R. ElShawi and S. Sakr, "Tpe-autoclust: A tree-based pipline ensemble framework for automated clustering," in *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*, 2022, pp. 1144–1153.
[28] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "Chacha for online automl," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 11 263–11 273. [Online]. Available: https://proceedings.mlr.press/v139/wu21d.html
[29] B. Celik, P. Singh, and J. Vanschoren, "Online automl: An adaptive automl framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.
[30] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Asml: A scalable and efficient automl solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning*, ser. Proceedings of Machine Learning Research, K. Eggensperger, R. Garnett, J. Vanschoren, M. Lindauer, and J. R. Gardner, Eds., vol. 256. PMLR, 09–12 Sep 2024, pp. 11/1–26. [Online]. Available: https://proceedings.mlr.press/v256/verma24a.html
[31] ——, "Auto-reg: A dynamic automl framework for streaming regression," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2025, pp. 245–256.
[32] Y. Liu, S. Li, and W. Tian, "Autocluster: Meta-learning based ensemble method for automated unsupervised clustering," in *Advances in Knowledge Discovery and Data Mining (PAKDD 2021)*. Springer, 2021, pp. 246–258.
[33] M. C. da Silva, G. M. Tavares, E. Medvet, and S. B. Junior, "Problem-oriented automl in clustering," *arXiv preprint arXiv:2409.16218*, 2024, arXiv:2409.16218.
[34] X. Wang, Z. Wang, Z. Wu, S. Zhang, X. Shi, and L. Lu, "Data stream clustering: An in-depth empirical study," *Proc. ACM Manag. Data*, vol. 1, no. 2, 2023.
[35] O. Arbelaitz, I. Gurrutxaga, J. Muguerza, J. M. Pérez, and I. Perona, "An extensive comparative study of cluster validity indices," *Pattern recognition*, vol. 46, no. 1, pp. 243–256, 2013.
[36] Y. Liu, Z. Li, H. Xiong, X. Gao, and J. Wu, "Understanding of internal clustering validation measures," in *2010 IEEE International Conference on Data Mining*, 2010, pp. 911–916.
[37] V. M. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1805–1858, 2020.
[38] D. Dua and C. Graff, "Uci machine learning repository," 2019. [Online]. Available: http://archive.ics.uci.edu/ml
[39] M. Duarte and Y. H. Hu, "Vehicle classification in distributed sensor networks," *Journal of Parallel and Distributed Computing*, vol. 64, no. 7, pp. 826–838, 2004.
[40] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "Moa: Massive online analysis," *The Journal of Machine Learning Research*, vol. 11, pp. 1601–1604, 2010.
[41] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine learning research*, vol. 7, no. Jan, pp. 1–30, 2006.
[42] P. Gijsbers, M. L. Bueno, S. Coors, E. LeDell, S. Poirier, J. Thomas, B. Bischl, and J. Vanschoren, "Amlb: an automl benchmark," *Journal of Machine Learning Research*, vol. 25, no. 101, pp. 1–65, 2024.
[43] H. Mühlenbein and G. Paaß, "From recombination of genes to the estimation of distributions I. Binary parameters," in *Proceedings of the 4th International Conference on Parallel Problem Solving from Nature (PPSN IV)*, ser. Lecture Notes in Computer Science, vol. 1141. Springer, 1996, pp. 178–187.
[44] J. Branke, "Memory enhanced evolutionary algorithms for changing optimization problems," in *Proceedings of the 1999 IEEE Congress on Evolutionary Computation (CEC)*, vol. 3. IEEE, 1999, pp. 1875–1882.
[45] Y. Jin and J. Branke, "Evolutionary optimization in uncertain environments—A survey," *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 3, pp. 303–317, 2005.
[46] S. Droste, "Analysis of the (1+1) EA for a dynamically changing OneMax-variant," in *Proceedings of the 2002 IEEE Congress on Evolutionary Computation (CEC)*, vol. 1. IEEE, 2002, pp. 55–60.

APPENDIX A
LABEL-FREE SELECTION VALIDITY

Figure 9 validates that *AutoDSC*'s label-free internal quality measure tracks external quality. Bucketing stages by the incumbent's multi-CVI score tercile shows that external ARI rises consistently with the internal score on PenDigits, and that on Sudden the incumbent already attains near-perfect quality across all terciles. Additionally, although *AutoDSC* never observes the true number of clusters, the incumbent's mean per-stage cluster count closely matches the ground truth on both streams, indicating that the label-free selection mechanism implicitly recovers the correct cluster structure.

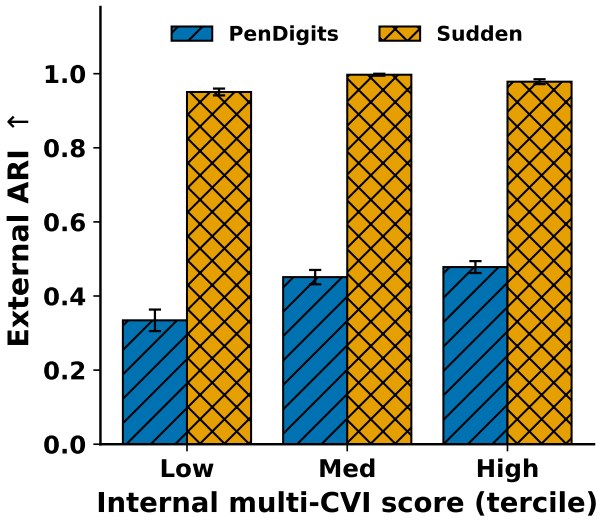


Fig. 9. Label-free selection validity on PenDigits and Sudden. External ARI rises with the incumbent's internal multi-CVI score, confirming the score tracks true quality.

APPENDIX B
SEARCH OVERHEAD DECOMPOSITION

Figure 10 decomposes *AutoDSC*'s per-stage wall-clock into base-clusterer training and prediction, multi-CVI scoring, and

the PGSSO search. The three components span several orders of magnitude on a log scale, with base-clusterer training dominating and the AutoML search machinery contributing a negligible fraction of the total time. This confirms that the cost of automation is dwarfed by the clustering it automates, explaining *AutoDSC*'s competitive throughput despite continuously searching the configuration space.

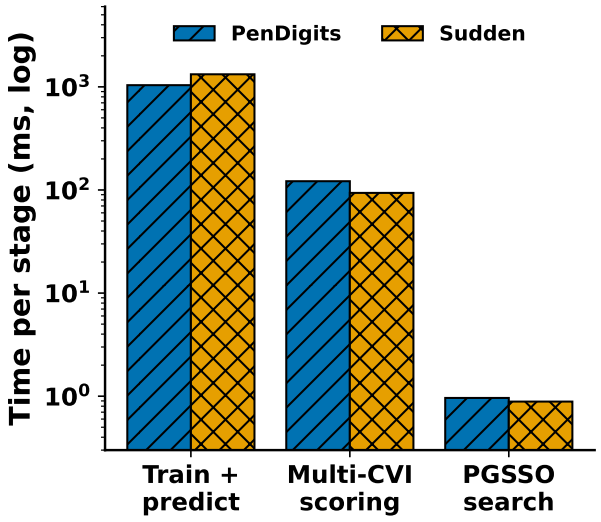


Fig. 10. Per-stage search overhead decomposition on log scale. PGSSO search is negligible relative to base-clusterer training.

Appendix C
Dimensionality Scalability

Figure 11 shows *AutoDSC*'s behaviour as feature dimensionality grows on a controlled synthetic stream. Clustering quality remains high and stable across the full dimensionality range, while throughput decreases only gradually, with no collapse in ARI or CMM at higher dimensions. This indicates that *AutoDSC*'s population-guided search scales gracefully with dimensionality and does not suffer from a curse-of-dimensionality effect on clustering quality.

Appendix D
Theoretical Analysis of PGSSO

We connect PGSSO's two core mechanisms to established theory in Estimation of Distribution Algorithms (EDAs) and evolutionary dynamic optimisation, justifying its behaviour under both stationary conditions and concept drift.

Population search formulation. PGSSO maintains a population $\mathcal{A}_t$ of K configurations, evaluates all members each stage using the quality score $\mu_t(\theta)$ (Section IV), and each stage generates n_{gen} candidate configurations, the elite-guided fraction by sampling a Gaussian $\mathcal{N}(m_j, \sigma_j^2)$ fitted to the elite archive $\mathcal{E}_t$ and the remainder uniformly at random. The elite-guided step mirrors one iteration of an estimation-of-distribution algorithm (EDA) [43]. Let $\theta_t^\star = \arg\max_{\theta\in\Theta}\ \mu_t(\theta)$ denote the global stage-optimal configuration and $\theta_t^* = \arg\max_{\theta_i\in\mathcal{A}_t}\ \mu_t(\theta_i)$ the incumbent selected

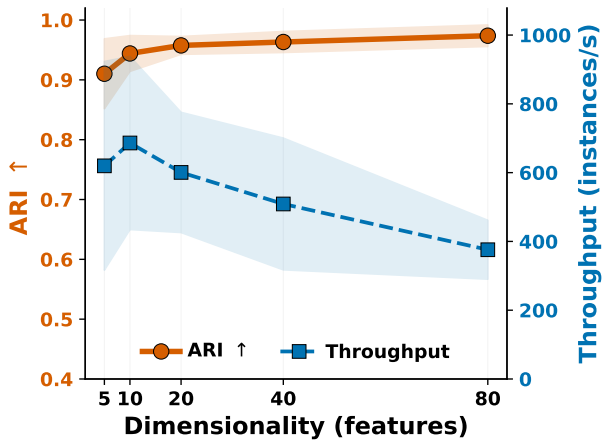


Fig. 11. Dimensionality scalability of AutoDSC. ARI remains stable while throughput degrades gracefully as feature count grows.

from the population. The per-stage quality gap is $\delta_t = \mu_t(\theta_t^\star) - \mu_t(\theta_t^*) \geq 0$.

Proposition 1 (Stationary concentration). When the data distribution is stationary, $\theta_t^\star = \theta^\star$ is fixed. PGSSO's elite-guided sampling fits $\mathcal{N}(m_j, \sigma_j^2)$ to the top-$e_{\max}$ configurations by recency-weighted score, the maximum-likelihood model-fitting step shared by estimation-of-distribution and evolution-strategy methods [23], [43]. Under persistent selection pressure such updates concentrate the sampling distribution on the highest-scoring region observed so far [43], so the chance of drawing a configuration within ε of $\theta^\star$ rises as elites accumulate near it. Lowering α_t when $\Delta\mu_t > 0$ (Section IV) sharpens this concentration, while the floor $\alpha_t \geq 0.1$ retains a residual stream of random candidates that keeps the search from locking onto a suboptimal basin. Because diversity pruning always retains the incumbent (elitism) and this residual exploration reaches every region of Θ with positive probability, the best-so-far score is non-decreasing and $\delta_t \to 0$ over a long stationary segment, so the population settles on a near-optimal configuration in a bounded number of stages.

Proposition 2 (Drift recovery). After a concept drift event that shifts $\theta_t^\star$, PGSSO must track the moving optimum, the central problem of evolutionary dynamic optimisation [44], [45]. Branke [44] shows that handling a changing optimum requires both retaining useful past structure and continuously injecting new candidates. PGSSO embodies both: when $\Delta\mu_t \leq 0$, α_t increases immediately (Section IV), raising the share of fresh random configurations among the n_{gen} generated each stage, while the recency-weighted score $s_i/(1+\Delta_i)$ demotes stale elites with large Δ_i to free archive slots for current ones. Farthest-first diversity pruning of $\mathcal{A}_t$ additionally prevents premature convergence to a no-longer-optimal region. Droste [46] proves that even a $(1{+}1)$ evolutionary algorithm tracks a moving optimum in expected polynomial time, provided the optimum drifts slowly enough relative to the mutation rate; PGSSO evaluates K configurations per stage rather than one, covering more of the search space between

drift events and so tolerating faster drift.

Corollary (Time-averaged quality bound). Within a stationary segment, the expected gap decays geometrically, $\mathbb{E}[\delta_t] \leq (1-p^\star)^t \cdot \delta_0$, where δ_0 is the gap at the start of the segment and $p^\star > 0$ is the per-stage probability of sampling within ε of $\theta^\star$; the latter is positive because the residual random component ($\alpha_t \geq 0.1$) reaches every region of the bounded space Θ with positive probability, and once such a configuration is drawn, elitism retains it. Assume each drift event is followed by a recovery period of at most C_{rec} stages, during which $\delta_t \leq \delta_{\max}$, after which the geometric decay resumes. Over T stages with D drift events of severity at most $\delta_{\max}$, the expected time-averaged gap is then bounded by

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[\delta_t] \;\leq\; \frac{D \cdot C_{\mathrm{rec}} \cdot \delta_{\max}}{T} \;+\; (1-p^\star)^{L_{\min}} \cdot \delta_0, \quad (8)$$

where $L_{\min}$ is the minimum stationary segment length, so the geometric residual $(1-p^\star)^{L_{\min}}$ upper-bounds the steady-state gap reached within each segment. This is an order bound rather than a tight estimate, but it isolates the operative trade-off: when drift is sparse ($D/T \to 0$) and stationary segments are long ($L_{\min} \to \infty$), both terms vanish and the time-averaged gap approaches zero, consistent with diversity being the key enabler of sustained quality under drift [45] and with the empirical results of Section VI.